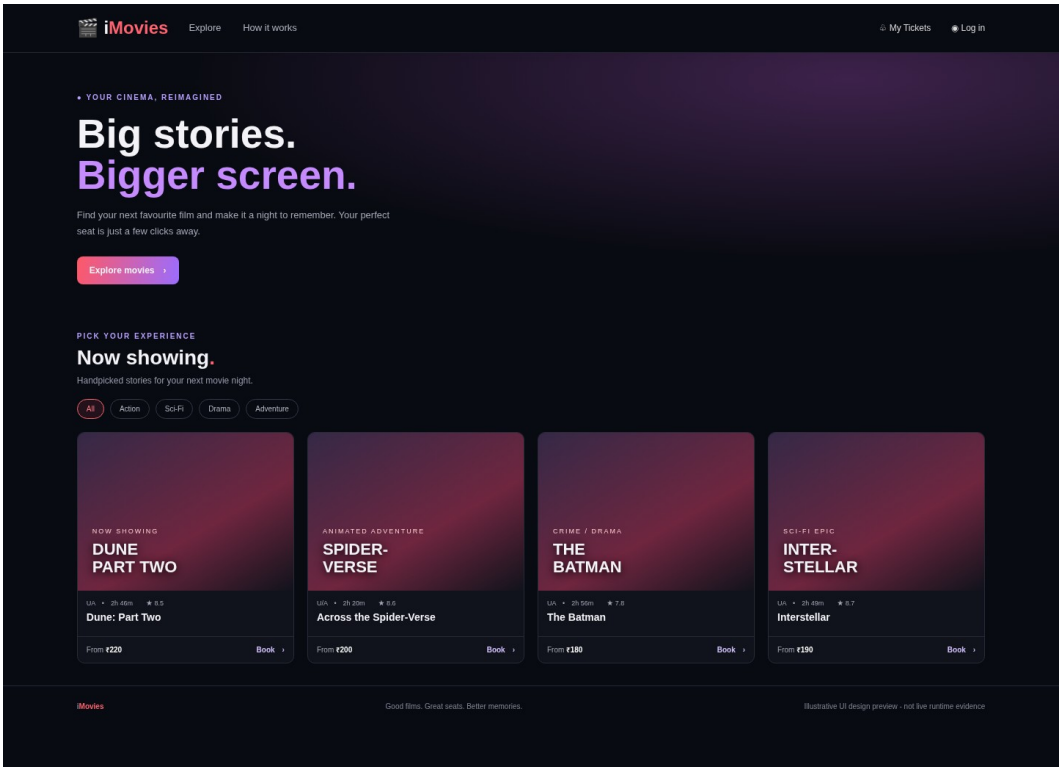
IMOVIES

MOVIE TICKET BOOKING MADE SIMPLE

Full Stack Developer - MERN | Intermediate Project



Illustrative UI design preview (not a screenshot of a running instance).

Submission detail	Value
Student name	Amay Gupta
Enrollment number	EN24CS3010116
Project title	IMovies - Movie Ticket Booking Made Simple
Technology stack	MongoDB, Express.js, React.js, Node.js
Report version	1.0 - Initial MVP documentation
Submission links	To be added after publishing the repository and recording the live demo

Document Guide

This report describes the current codebase and its intended behaviour. It follows the supplied template structure: project description and scenario, system requirements, architecture, project setup, backend, database, frontend, execution, output evidence, and final deliverables.

Section	Contents
1	Project description, scenario, objectives and scope
2	System requirements
3	Project architecture, ER model, user flow, features, roles and MVC/request handling
4	Project setup and configuration
5	Backend development and API endpoints
6	Database design and seed data
7	Frontend development
8	Execution guide
9	Test plan and output evidence
10	Limitations, future enhancements and references

Evidence status: the diagrams are architecture documentation and the UI images are illustrative previews. They are not proof of a live test. The app must be started on the submitter's computer, tests must be recorded, and actual output screenshots/demo links added before final hand-in.

1. Project Description

IMovies - Movie Ticket Booking Made Simple is a full-stack web application that helps users discover movies and make a demo ticket booking through a single interface. The React frontend displays movie information, search and genre filters, show dates and times, a seat-selection layout, and a My Tickets view. The Express.js backend exposes REST API endpoints for movies, user registration/login, and booking operations. MongoDB stores user, movie, and booking documents, while Mongoose defines schemas and references between collections. Passwords are hashed using bcryptjs and protected requests use JSON Web Tokens (JWT). The current MVP does not process payments and should be used for learning and demonstration rather than real-world ticket sales.

1.1 Scenario

Moviegoers often need to compare films, decide on a showtime, select seats and keep booking details accessible. IMovies demonstrates this flow in one responsive web application. A visitor can browse available sample movies, search or filter by genre, open a movie, select a date/time and seats, then sign in to submit the booking. The backend checks the selected seats against existing confirmed bookings for the same movie, date and showtime, stores the booking, and returns a confirmation code. The user can later view their saved bookings in My Tickets.

1.2 Problem Statement

The project provides a learning-focused example of how a modern web client communicates with a REST backend and persistent database to implement a multi-step booking workflow. It also demonstrates user authentication, data modeling, input validation, and the separation of UI and server-side responsibilities.

1.3 Objectives

- Build a clean responsive movie catalogue with search and genre filters.

- Implement user registration and login with password hashing and JWT authentication.
- Represent movies, users and bookings with MongoDB/Mongoose models.
- Allow a signed-in user to choose a movie, show date/time and multiple seats.
- Check already-booked seats on the server before creating a new booking.
- Provide a My Tickets screen and clear local setup/test documentation.

1.4 Project Scope

Included in the MVP	Not included in this version
Movie catalogue with sample records	Real payment gateway or refunds
Register/login and protected routes	Admin dashboard and movie CRUD interface
Seat selection and demo booking confirmation	Atomic reservation/locking for concurrent booking attempts
My Tickets history	Real cinema inventory integrations or email/SMS delivery
Local development setup with MongoDB	Production hosting, monitoring and hardened security

2. System Requirements

2.1 Software Requirements

Software/tool	Purpose
Windows 10/11, macOS or Linux	Development environment
Node.js 18+ and npm	JavaScript runtime and package management
React + Vite	Client UI and local development server
Express.js	REST API routing and request handling
MongoDB Community Server or MongoDB Atlas	Persistent database
Mongoose	MongoDB schema modeling and queries
Visual Studio Code	Recommended code editor and terminal
Git and GitHub	Version control and project submission
Google Chrome / Firefox	Browser testing
Postman (optional)	Manual API request testing

2.2 Hardware Requirements

Component	Minimum recommendation	Purpose
Processor	Intel Core i5 (8th Gen) / AMD Ryzen 5 or similar	Development, IDE and browser use
RAM	8 GB minimum; 16 GB recommended	Run editor, server, database and browser together
Storage	At least 1 GB free for source and packages; more for tools	Dependencies, database files and documentation
Display	1366 × 768 or higher	Review UI and work in code editor
Internet	Required for package installation and remote images/Atlas	Dependency downloads and remote services

3. Epic 1 - Project Architecture

The application uses a client-server architecture. The React/Vite client sends HTTP requests to an Express API. Public movie routes return movie data; registration and login routes create or validate accounts; booking routes require a JWT and read/write booking data through Mongoose models. MongoDB stores the persistent documents.

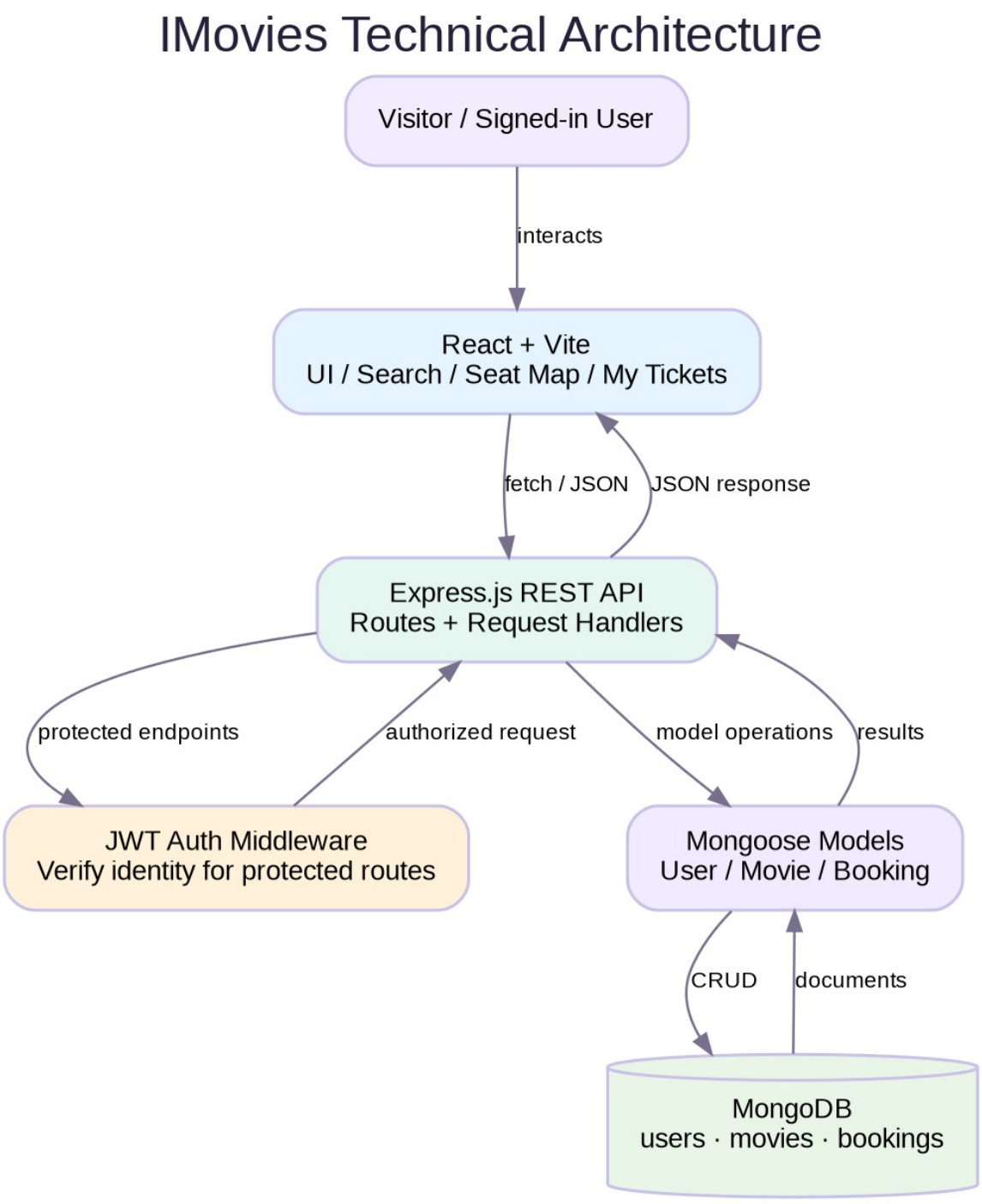


Figure 1. Technical architecture of the IMovies MVP.

3.1 Architecture Components

Component	Responsibilities
React + Vite client	Renders pages/components, movie cards, search/filter controls, authentication forms, seat selection and My Tickets; calls the REST API using fetch.
Express API	Exposes `/api/health`, `/api/movies`, `/api/auth` and `/api/bookings` endpoints; parses JSON; applies CORS; returns JSON responses.
Authentication middleware	Reads the Bearer token, verifies JWT and attaches the signed-in user to protected requests.
Mongoose models	Define User, Movie and Booking fields, validation rules, references and timestamps.
MongoDB	Persists account, movie and booking documents between sessions.

3.2 ER Diagram / Data Model

MongoDB is a document database, so the model is implemented as collections with ObjectId references rather than fixed relational tables. A user can have multiple bookings, and a movie can be associated with multiple bookings. Each booking references one user and one movie.

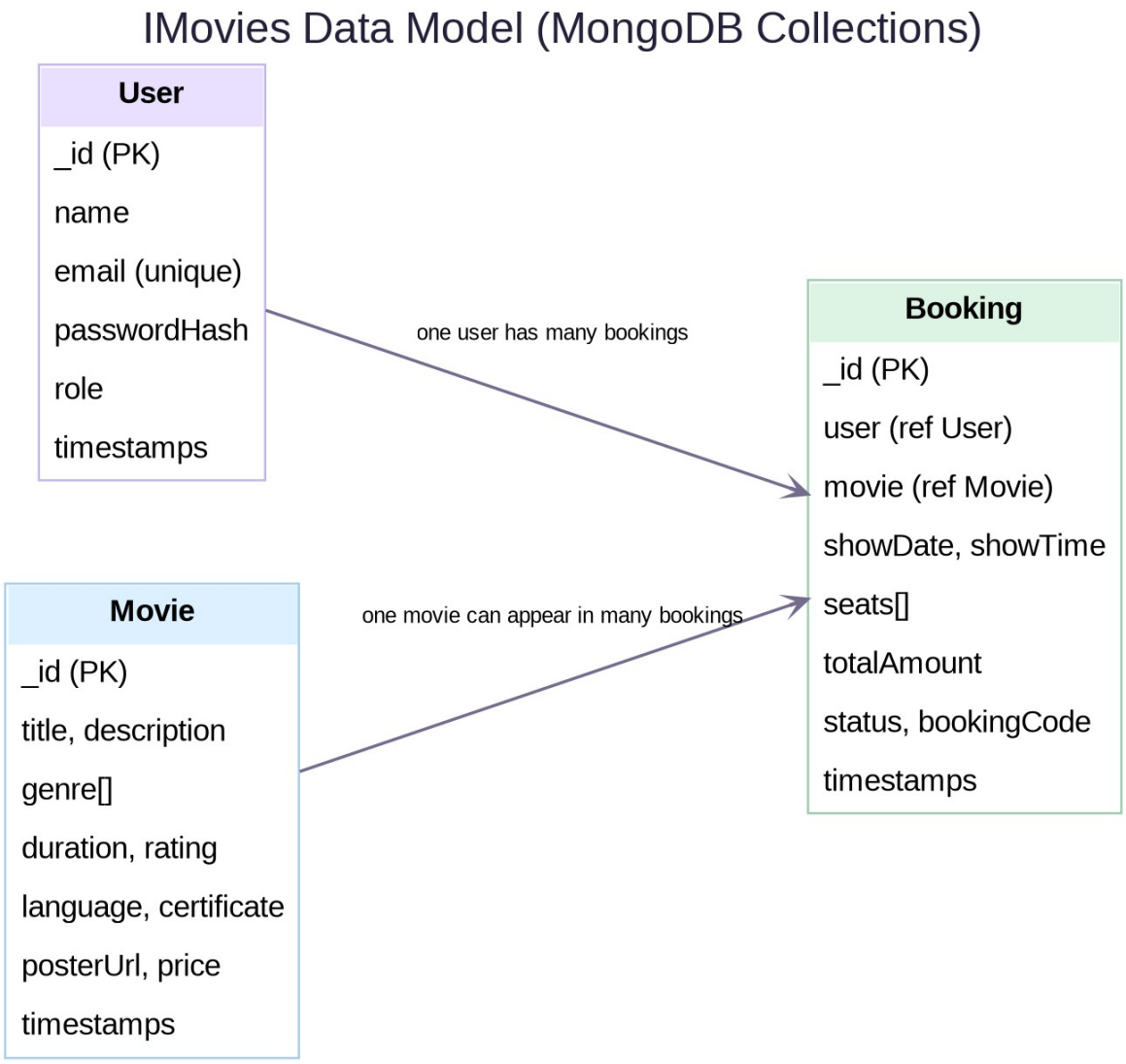


Figure 2. Collection relationships. PK = document identifier; references connect booking documents to users and movies.

3.3 Collection Summary

Collection	Important fields	Relationship / constraints
User	name, email, passwordHash, role, timestamps	Email is unique; passwordHash contains the hash rather than the raw password.
Movie	title, description, genre[], duration, rating, language, certificate, posterUrl, price	Movie catalogue record; price is used to calculate the demo booking total.
Booking	user, movie, showDate, showTime, seats[], totalAmount, status, bookingCode, timestamps	References User and Movie; unique bookingCode; status is confirmed or cancelled.

3.4 User Flow

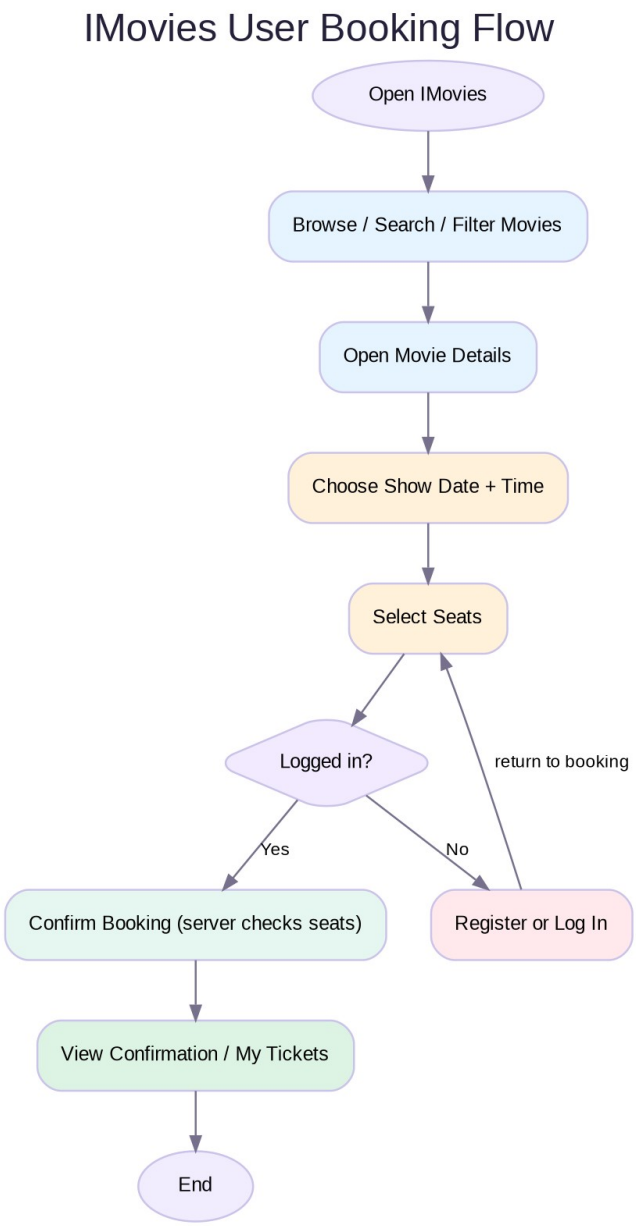


Figure 3. Main user booking flow. If authentication is required, the user can sign in and return to the booking modal to confirm.

3.5 Features

- Browse seeded movie cards with genre, runtime, rating, language/certificate and starting ticket price.
- Search movie titles and filter by one genre.
- Register an account and sign in with email/password.
- Choose a date and one of the demo showtime choices.
- Select multiple seat labels from the seat map and see a calculated total.
- Submit booking requests with JWT; server validates basic seat format and rejects seats already stored in confirmed bookings for the selected movie/date/showtime.
- Review saved bookings and their confirmation codes under My Tickets.
- Show helpful messages for common validation or API failures.

3.6 Roles and Responsibilities

Role	Current responsibilities
Visitor	Browse/search/filter movies and inspect movie details; authentication is required before a booking is saved.
Registered user	Log in, create bookings and view only their own booking history.
Application/API	Validate request data, verify JWT, calculate total price, check existing seats and return responses.
Developer / maintainer	Configure environment variables, seed sample data, run tests, update code and protect secrets.
Administrator	No administrative UI is implemented in this MVP; the `role` field exists for future extension.

3.7 MVC / Request-Handling Pattern

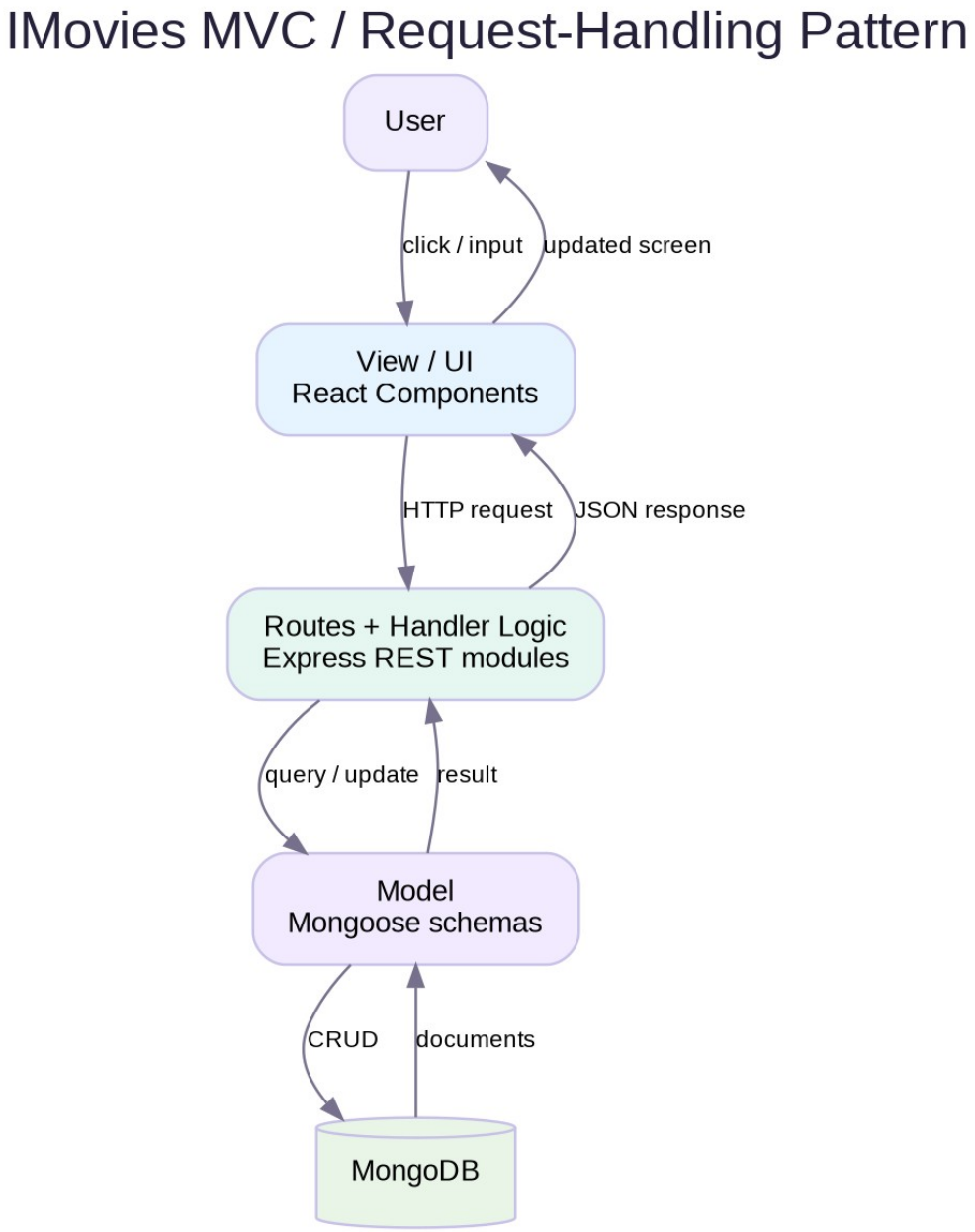


Figure 4. Adapted MVC/request-handling view of the current implementation.

Model: Mongoose schemas in `server/models/` define User, Movie and Booking data. Controller/request-handler logic: asynchronous route handlers in `server/routes/` validate requests and coordinate model operations. View: the user-facing view is implemented in React under `client/src/`; the backend itself returns JSON rather than rendering server-side HTML. This is an MVC-inspired separation, although the current MVP keeps controller logic inside route modules rather than maintaining a separate controllers folder.

3.8 Security and Validation Notes

- Passwords are hashed before being saved; the plaintext password is not stored.
- JWT authentication is required for booking creation and retrieving the signed-in user's bookings.

- The booking endpoint validates required fields, duplicate seats and seat-label format.
- Never commit `.env` files or secrets; change the sample JWT secret before any hosted deployment.
- Known risk: availability check and booking insert are separate operations, not an atomic transaction. Two concurrent requests could race. Real ticketing needs transactional/unique reservation logic, rate limiting and broader input validation.

4. Epic 2 - Project Setup and Configuration

4.1 Folder Structure

```
IMovies/
├── client/
│   ├── src/App.jsx
│   ├── src/main.jsx
│   ├── src/styles.css
│   ├── index.html
│   └── package.json
├── server/
│   ├── data/movies.json
│   ├── middleware/auth.js
│   ├── models/User.js
│   ├── models/Movie.js
│   ├── models/Booking.js
│   ├── routes/auth.js
│   ├── routes/movies.js
│   ├── routes/bookings.js
│   ├── index.js
│   ├── seed.js
│   └── package.json
├── docs/
└── assets/
```

4.2 Backend Setup

1. Install Node.js 18 or later and make sure `node --version` and `npm --version` work.
2. Start MongoDB locally or create an Atlas database and copy its connection URI.
3. Open a terminal in the `server` folder and copy `.env.example` to `.env`.
4. Set `MONGO_URI` and a private, long random `JWT_SECRET` in `server/.env`.
5. Run `npm install`, then `npm run seed` to insert six sample movie records, then `npm run dev`.

```
cd server
# create .env from .env.example and edit values
npm install
npm run seed
npm run dev
```

4.3 Frontend Setup

6. Open a second terminal in the `client` folder and copy `.env.example` to `.env`.
7. Keep `VITE_API_URL=http://localhost:8000/api` for local backend use.
8. Run `npm install` and `npm run dev`, then open the Vite URL, usually `http://localhost:5173`.

```
cd client
# create .env from .env.example
npm install
```

```
npm run dev
```

4.4 Environment Variables

File / variable	Meaning	Safe handling
server/.env: PORT	Port for Express API (default 8000)	Local configuration
server/.env: MONGO_URI	MongoDB connection URI	Do not commit; may contain credentials
server/.env: JWT_SECRET	Secret used to sign/verify JWTs	Use a strong private value; do not commit
server/.env: CLIENT_URL	Allowed browser origin for local CORS	Restrict to known origins in production
client/.env: VITE_API_URL	Base URL for the API	Public client configuration only; never store secrets here

5. Epic 3 - Backend Development

The backend entry point is `server/index.js`. It loads environment configuration, connects to MongoDB, enables JSON parsing and CORS, mounts route modules, and defines a common error response. It listens on port 8000 by default after a successful database connection.

5.1 API Endpoint Summary

Method	Endpoint	Purpose	Access
GET	/api/health	Return API status	Public
GET	/api/movies	List catalogue; optional search and genre query	Public
GET	/api/movies/:id	Fetch one movie by ID	Public
POST	/api/auth/register	Create account and return token	Public
POST	/api/auth/login	Validate credentials and return token	Public
GET	/api/auth/me	Return current user profile	JWT
POST	/api/bookings	Validate seats and create booking	JWT
GET	/api/bookings/my	List bookings belonging to current user	JWT

5.2 Typical Booking Request

```
POST /api/bookings
Authorization: Bearer <JWT>
Content-Type: application/json

{
  "movieId": "<MongoDB movie id>",
  "showDate": "2026-10-24",
  "showTime": "4:30 PM",
  "seats": ["C3", "C4"]
}
```

The API validates that a movie, date, showtime and at least one seat are supplied; checks for duplicate/invalid seat labels; loads the movie; queries bookings for already-confirmed selected seats for the same movie, date and time; calculates `movie.price × number of seats`; and returns the created booking and confirmation code. This is a demo flow and is not a replacement for a transaction-safe production ticket inventory system.

5.3 Error Responses

- 400 Bad Request: required fields missing, repeated seats or invalid seat labels.
- 401 Unauthorized: missing, invalid or expired JWT for protected routes.
- 404 Not Found: movie ID does not resolve to a movie.
- 409 Conflict: duplicate email during registration or one or more seats are already booked.
- 500 Internal Server Error: unexpected server or database error.

6. Epic 4 - Database Development

6.1 MongoDB and Mongoose

MongoDB stores documents in three collections. Mongoose schemas specify required fields, string trimming/lowercasing, enum values, references and timestamps. User passwords are stored as `passwordHash`. Booking records include an array of selected seats, total amount, status and a human-readable code.

6.2 Seed Dataset

`server/data/movies.json` contains six sample movie records with titles, descriptions, genres, durations, ratings, languages/certificates, poster URLs and ticket prices. `server/seed.js` loads this JSON file and inserts the records into MongoDB. It is suitable for local demonstration only. Running the seed script again deletes and replaces existing movie documents, so do not run it against a database containing records you need to retain.

Sample record	Genres	Duration	Demo price
Dune: Part Two	Sci-Fi, Adventure	166 min	₹220
Spider-Man: Across the Spider-Verse	Animation, Action	140 min	₹200
The Batman	Action, Crime	176 min	₹180
Interstellar	Sci-Fi, Drama	169 min	₹190
Jawan	Action, Thriller	169 min	₹180
12th Fail	Drama	147 min	₹160

6.3 Relationship Integrity and Limitations

Bookings reference both a user and a movie by ObjectId. The “my bookings” endpoint filters by the authenticated user ID, so one user does not receive another user’s booking list from this endpoint. The current code checks previously booked seats but does not make the check-and-insert sequence atomic; this limitation must be addressed before production use.

7. Epic 5 - Frontend Development

The frontend is a single-page React application bootstrapped with Vite. `App.jsx` manages the movie catalogue, filters, selected movie, seat selections, authentication modal, booking modal and ticket list. `styles.css` contains the responsive dark-cinema design, and Lucide React provides interface icons.

7.1 Main Interface Areas

- Top navigation with brand, Explore, How it works, My Tickets and login/account controls.
- Hero section introducing the booking experience.
- Movie catalogue cards with metadata, ratings, starting prices, search input and genre chips.

- Booking modal with date picker, showtime selector, seat map, selected seat count and total price.
- Login/register modal for account access.
- My Tickets modal with booking code, date/time, seats and total.

7.2 UI Preview - Home

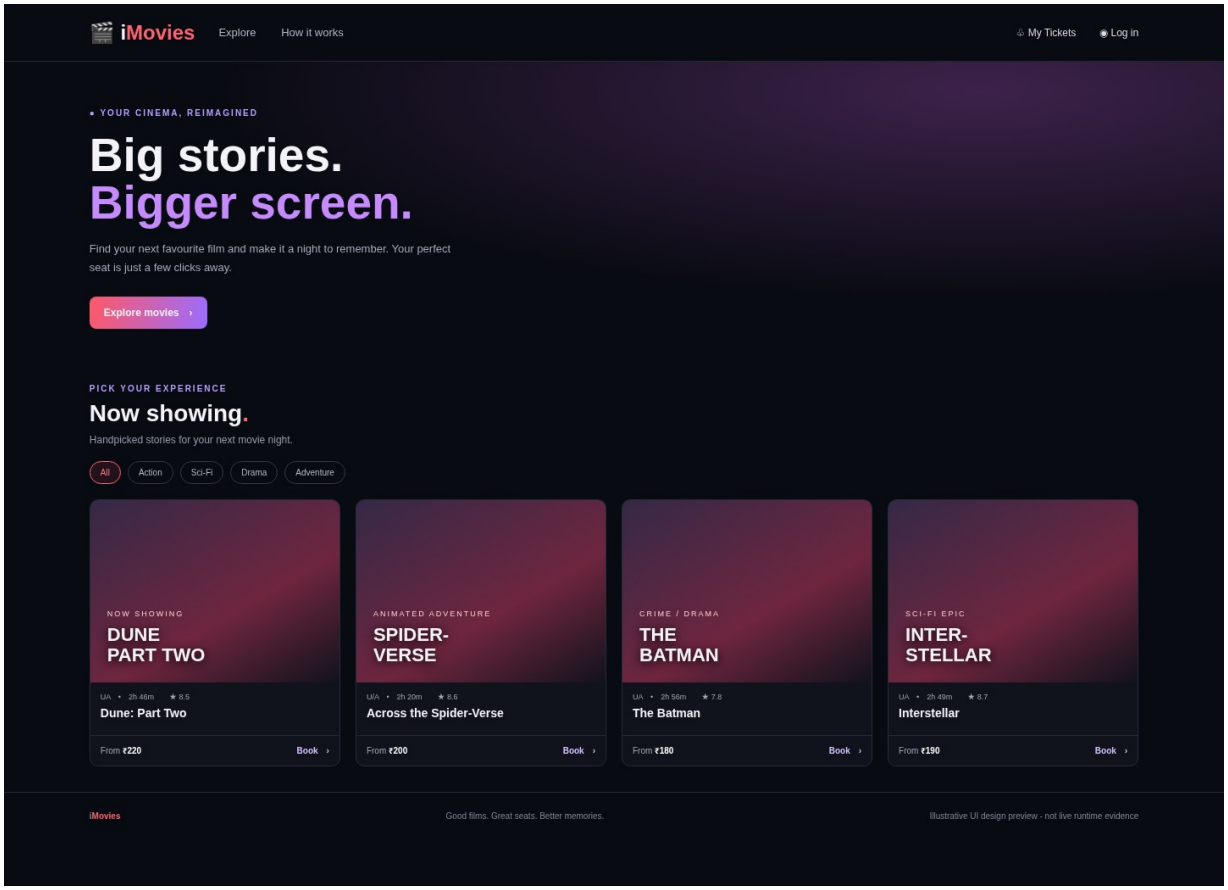


Figure 5. Illustrative home/catalogue layout. This image was composed as a design preview, not captured from the running React app.

7.3 UI Preview - Seat Selection

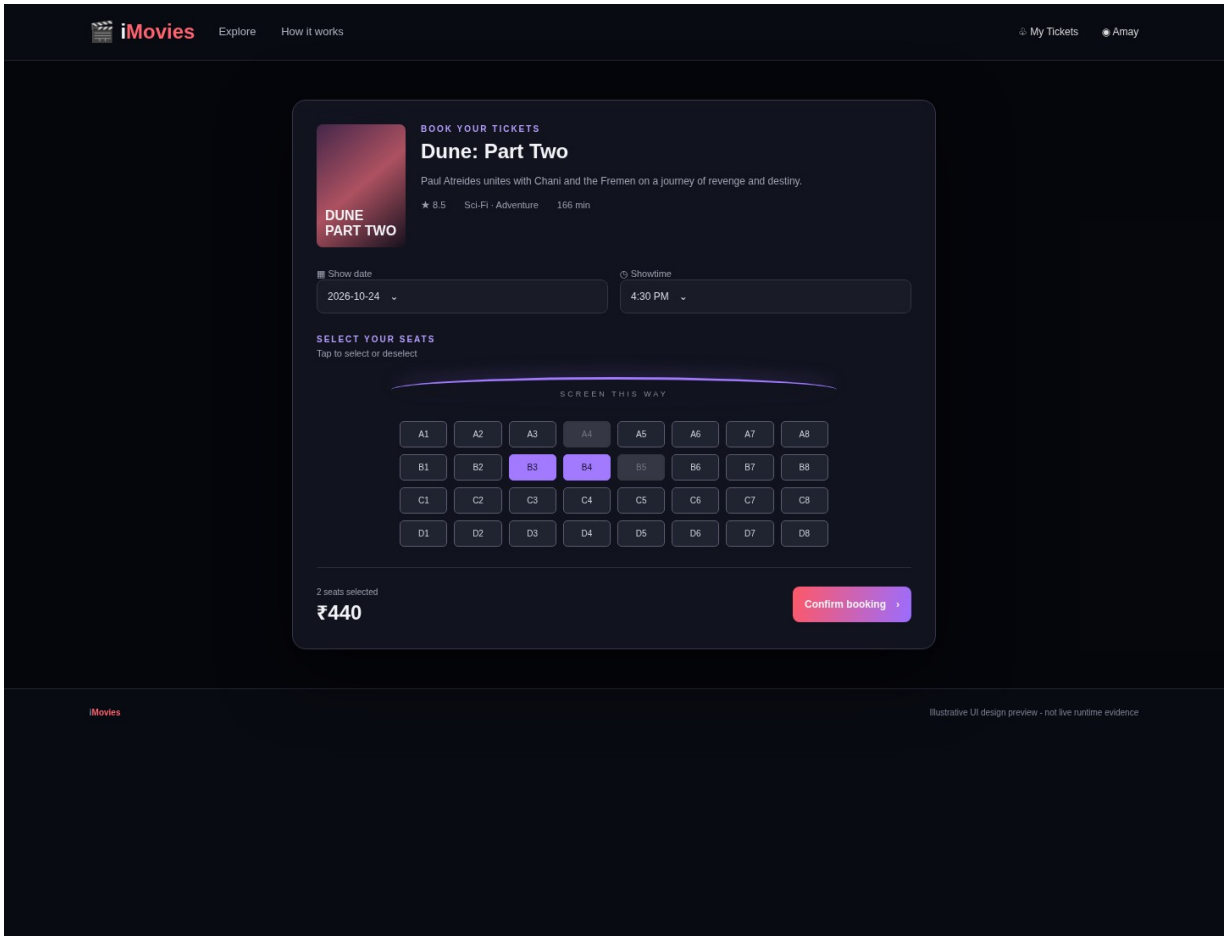


Figure 6. Illustrative booking/seat-selection layout. Seat values shown are examples only.

7.4 UI Preview - My Tickets

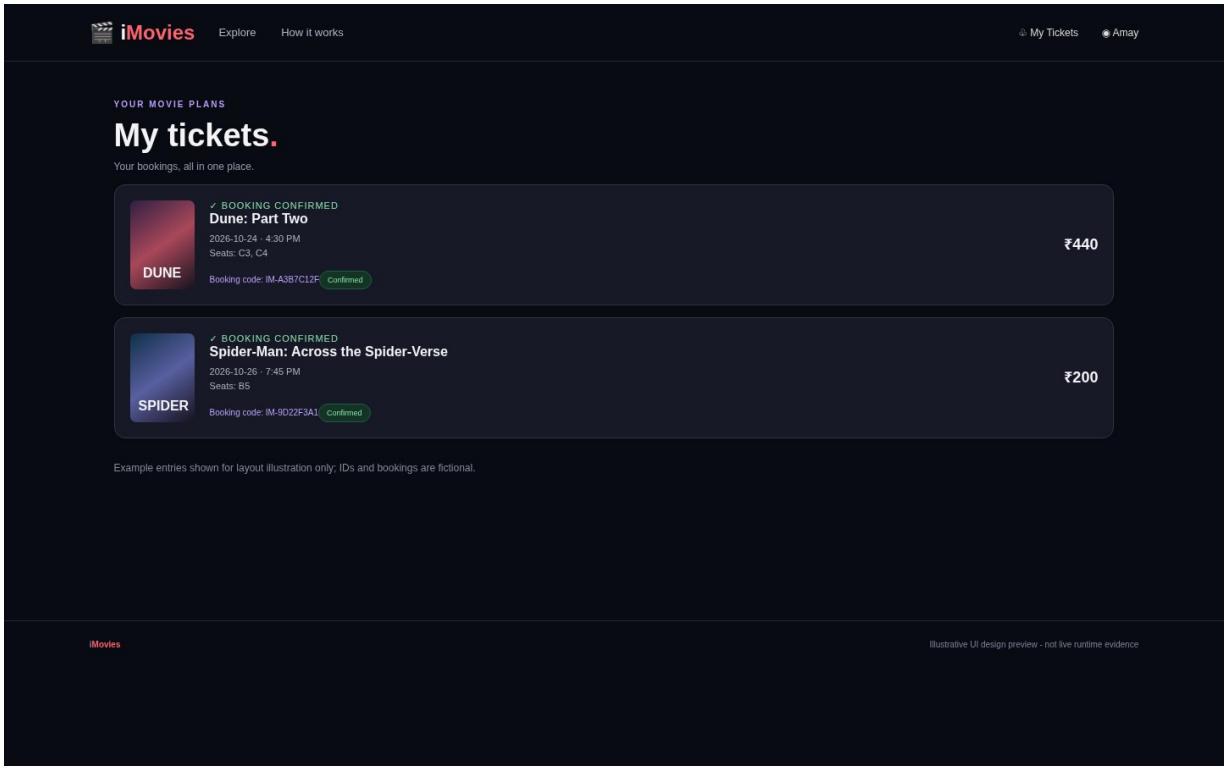


Figure 7. Illustrative My Tickets layout; sample booking codes and entries are fictional.

8. Epic 6 - Project Execution

8.1 Start the Application

9. Ensure MongoDB is running and the server `.env` points to the correct database.
10. In terminal 1, navigate to `server/`, run `npm install` once, then `npm run seed` and `npm run dev`.
11. Check `http://localhost:8000/api/health` returns status `ok`.
12. In terminal 2, navigate to `client/`, create `.env` from `.env.example`, run `npm install` once and `npm run dev`.
13. Open the Vite development URL (usually `http://localhost:5173`) in Chrome or Firefox.

8.2 User Demonstration Steps

14. Browse and filter the movie list; try a title search.
15. Register a demo user with an email you can access, or sign in with an existing local test account.
16. Open a movie, select a future date and showtime, and choose seat labels.
17. Confirm the booking; if a selected seat was booked earlier for that movie/date/time, choose another seat.
18. Open My Tickets and verify the movie, date/time, seat list, booking code and total.

8.3 Troubleshooting

Symptom	Likely cause	Action
MongoDB connection failure	Service not started or URI incorrect	Start MongoDB; review `server/.env` and Atlas IP/network settings.
Frontend has no movies	Seed not run, API unavailable or browser cannot reach API	Run `npm run seed`; check backend terminal and `/api/health`.
CORS error	Client URL differs from server `CLIENT_URL`	Set the correct local origin in server `.env`, then restart backend.
401 on booking/My Tickets	User is logged out or token expired	Log in again and retry.
Poster image absent	External image URL or internet connection unavailable	Check connection; poster fallback placeholder may display.
Port already in use	Another local process occupies the port	Stop that process or choose a free port and update API URL as needed.

9. Testing and Output Evidence

Test execution needs a working Node/npm dependency installation and a reachable MongoDB instance. In the environment used to package this report, the npm dependency install attempt did not finish, and a live MongoDB service was not available. Therefore, no end-to-end pass claim is made here. The manual cases in `docs/TEST_PLAN.md` must be executed and the actual status/results filled in by the project owner.

9.1 Test Matrix

Test group	What to verify	Status in this package
Static source review	Source folders and configuration samples exist	Packaged; Node syntax checks passed for server JS
API availability	Health endpoint starts successfully	Not run live
Database integration	Seed and movie list read from MongoDB	Not run live
Authentication	Register/login/JWT protected routes	Not run live
Booking logic	Seat validation, duplicate-seat conflict and totals	Not run live
Frontend	Search/filter, modal flow and My Tickets	Not run live
Cross-browser/responsive	Layout on target desktop/mobile browsers	Not run live

9.2 Required Real Screenshots

Before final submission, capture screenshots directly from the running app and save them in `assets/screenshots/`. Suggested screenshots: (1) home/catalogue; (2) search or genre filter; (3) register/login; (4) seat-selection modal; (5) My Tickets showing a real local demo booking; (6) API health response. The design previews in this report should be replaced with real output screenshots before treating the report as final evidence.

9.3 Demo Video

Use the timed steps in `docs/DEMO_SCRIPT.md`. Record the actual website after successful test execution, upload to a course-approved share location, test that the link opens for a reviewer, and paste the real URL into the submission worksheet. No demo video has been fabricated or uploaded as part of this package.

10. Limitations and Future Enhancements

10.1 Current Limitations

- No online payment provider or refund flow.
- No admin UI for creating/updating/removing movies or showtimes.
- Seat availability check can race under simultaneous requests and is not safe as a real inventory reservation system.
- The seat map includes fixed preview-occupied positions; only persisted backend bookings are authoritative during submission.
- Basic validation is implemented, but production hardening (rate limiting, deeper validation, transaction strategy, secure deployment config and monitoring) remains.
- Poster URLs depend on external hosts and can change.
- Live integration and browser testing still need to be performed on the submitter’s computer.

10.2 Future Enhancements

- Admin dashboard for movies, theatre screens, shows and pricing.
- Transactional or atomic seat reservation with expiry/locking.
- Payment gateway integration and verified payment callbacks.
- Email ticket delivery and downloadable ticket PDF/QR code.
- Pagination, loading states, stronger form validation and accessibility audit.
- Automated unit/API/UI tests and CI pipeline.
- Deploy frontend, API and MongoDB with production environment variables and monitoring.

11. Conclusion

IMovies demonstrates the main building blocks of a MERN full-stack application: a React interface, Express REST API, Mongoose data models, MongoDB persistence, JWT authentication and a multi-step booking flow. This package includes the source project, setup documentation, architectural diagrams, test plan, demo outline and an editable report plus PDF. Completion of the actual submission still requires running the application against MongoDB, recording the test results, capturing real runtime screenshots, publishing the repository, recording/uploading the demo video and submitting the real links through SkillWallet.

12. References

- React. “Learn React.” https://react.dev/learn
- Vite. “Guide.” https://vite.dev/guide/
- Node.js. “Learn Node.js.” https://nodejs.org/en/learn
- Express.js. “Express documentation.” https://expressjs.com/
- MongoDB. “MongoDB Manual.” https://www.mongodb.com/docs/
- Mongoose. “Mongoose documentation.” https://mongoosejs.com/docs/
- JWT.io. “Introduction to JSON Web Tokens.” https://jwt.io/introduction
- npm. “bcryptjs package.” https://www.npmjs.com/package/bcryptjs
- Git. “Documentation.” https://git-scm.com/doc
- GitHub. “GitHub Docs.” https://docs.github.com/
- MDN Web Docs. “Web platform documentation.” https://developer.mozilla.org/
- Lucide. “Lucide React.” https://lucide.dev/guide/packages/lucide-react

Appendix A - Submission Links

Fill the following with real links after publishing. Do not submit the bracketed placeholder text as a URL.

Deliverable	Link / location
GitHub repository	[PASTE REAL GITHUB REPOSITORY URL]
Demo video	[PASTE REAL SHAREABLE VIDEO URL]
Code Drive link (if separately required)	[PASTE REAL DRIVE URL OR MARK NOT REQUIRED]
Report	docs/IMovies-Project-Report.pdf and editable DOCX
Source package	Client and server code at repository root
Test evidence	assets/screenshots/ after live local testing